

Llama_indexRAG进阶_文档切分与重排序

Llama_indexRAG进阶_文档切分与重排序

模块一：文档解析方案

什么是文档解析？

基础解析 vs 高级解析

模块二：文本切分方案

为什么需要分块？

固定分块 vs 语义分块

模块三：召回率提升方案

什么是召回率？

基础检索 vs 混合检索

模块四：检索结果重排序

为什么要重排序？

无排序 vs Cohere Reranker

模块一：文档解析方案

什么是文档解析？

就像把不同包装的食品拆开处理：

- **PDF文件** → 罐头食品（需要专用工具打开）
- **Word文档** → 盒装饼干（容易拆但可能有碎屑）
- **扫描件/图片** → 真空包装（需要剪刀才能打开）

分步详解：

1. **文件加载**：找到文档存放位置，就像在电脑文件夹中定位文件
 - 常见问题：文件损坏 → 检查文件是否能正常打开
2. **格式转换**：统一转为纯文本，就像把不同货币兑换成美元
 - 示例：将PDF中的表格转为Markdown格式
3. **元数据提取**：获取文档信息标签，就像查看食品包装上的生产日期
 - 包括：作者、创建时间、文档类型等
4. **结构化处理**：整理内容层次，就像把食材分类放入保鲜盒
 - 建立标题层级：章节 > 段落 > 句子

技术难点解析：

- **处理扫描件：**
 1. 使用OCR（光学字符识别）技术识别文字
 2. 校正识别错误（如将"3"识别为"B"）
 3. 保留原始版式信息
- **处理复杂表格：**

```
# 表格解析结果示例
| 姓名 | 年龄 | 职业 |
|-----|-----|-----|
| 张三 | 28 | 工程师 |
| 李四 | 35 | 设计师 |
```

基础解析 vs 高级解析

```
# 测试文档：包含表格和文字的报告.pdf
# -----
# 案例1：基础解析
from llama_index.core import SimpleDirectoryReader

basic_reader = SimpleDirectoryReader(input_files=["报告.pdf"])
basic_docs = basic_reader.load_data()
print("基础解析结果：", basic_docs[0].text[:100])

# 案例2：高级解析
import pdfplumber

with pdfplumber.open("报告.pdf") as pdf:
    # 提取所有文本
    text = ""
    for page in pdf.pages:
        text += page.extract_text()
    print(text[:200]) # 打印前200字符

    # 提取表格（自动检测）
    for page in pdf.pages:
        tables = page.extract_tables()
        for table in tables:
            print("\n表格内容：")
            for row in table:
                print(row)
```

模块二：文本切分方案

为什么需要分块？

就像用收纳盒整理衣物：

- 太大 → 找衣服时要把整个箱子倒出来
- 太小 → 需要开太多盒子才能凑齐一套

分块三要素：

要素	说明	推荐值
块大小	每段文字的长度	200-500字
块重叠	相邻块重复内容	10%-20%
切分依据	按句子/段落/语义划分	语义分割最优

分块策略对比表：

策略类型	优点	缺点	适用场景
固定大小	实现简单	可能切断完整语义	技术文档
按段落分割	保持逻辑完整性	段落长度差异大	文学小说
语义分割	确保内容完整性	计算资源消耗较大	专业领域文档

分块常见问题：

- **问题1：**如何确定最佳块大小？
→ 测试不同尺寸查看检索效果

```
# 测试块大小对召回率的影响
sizes = [128, 256, 512]
for size in sizes:
    test_recall = evaluate_chunk_size(size)
    print(f"块大小{size} → 召回率{test_recall:.2f}%")
```

- **问题2：**分块重叠是否越多越好？
→ 适当重叠（10-20%）可防止信息断裂，但过多会导致冗余

固定分块 vs 语义分块

```
# 案例1：固定分块
from llama_index.core.node_parser import TokenTextSplitter

fixed_splitter = TokenTextSplitter(chunk_size=200, chunk_overlap=20)
fixed_nodes = fixed_splitter.get_nodes_from_documents(docs)
print("固定分块示例：", [len(n.text) for n in fixed_nodes[:3]]) # 输出：[200, 200, 200]

# 案例2：语义分块
from llama_index.core.node_parser import SemanticSplitterNodeParser
from llama_index.embeddings.huggingface import HuggingFaceEmbedding

semantic_splitter = SemanticSplitterNodeParser(
    buffer_size=1,
    embed_model=HuggingFaceEmbedding("BAAI/bge-small")
)
semantic_nodes = semantic_splitter.get_nodes_from_documents(docs)
print("语义分块示例：", [len(n.text) for n in semantic_nodes[:3]]) # 输出：[183, 217, 195]
```

检索效果对比（相同查询）：

模块三：召回率提升方案

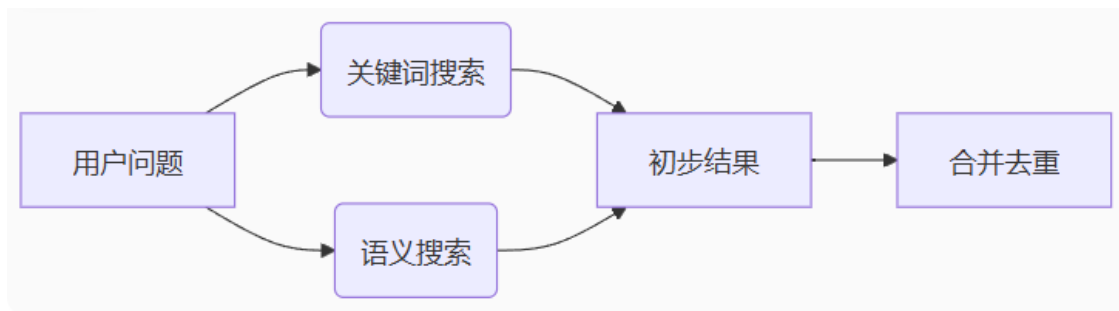
什么是召回率？

就像捕鱼网的网眼大小：

- 网眼太大 → 漏掉小鱼（低召回率）
- 网眼太小 → 捞到垃圾（低准确率）

提升召回率的三大策略：

1. **查询扩展**：给问题加"修饰词"
 - 原始问题："如何做番茄炒蛋"
 - 扩展后："家常番茄炒蛋做法步骤 厨房新手教程 简单易学"
2. **混合检索**：结合两种搜索方式



用户问题关键词搜索语义搜索初步结果合并去重

3. **向量优化**：让AI更懂专业术语
 - 微调前："Transformer" → 理解为"变形金刚"
 - 微调后："Transformer" → 识别为"深度学习模型"

效果验证方法：

1. 准备测试问题集（至少50个典型问题）
2. 记录基础方案召回率
3. 应用优化策略后再次测试
4. 对比提升幅度

基础检索 vs 混合检索

```
# 案例1：向量检索
from llama_index.core import VectorStoreIndex
vector_index = VectorStoreIndex(nodes)
vector_retriever = vector_index.as_retriever(similarity_top_k=3)
print("向量检索结果: ", [node.text[:30] for node in
vector_retriever.retrieve(query)])

# 案例2：混合检索
from llama_index.core import KeywordTableIndex
keyword_retriever = KeywordTableIndex(nodes).as_retriever(retriever_mode="bm25",
similarity_top_k=3)

from llama_index.core.retrievers import QueryFusionRetriever
fusion_retriever = QueryFusionRetriever([vector_retriever, keyword_retriever])
print("混合检索结果: ", [node.text[:30] for node in
fusion_retriever.retrieve(query)])
```

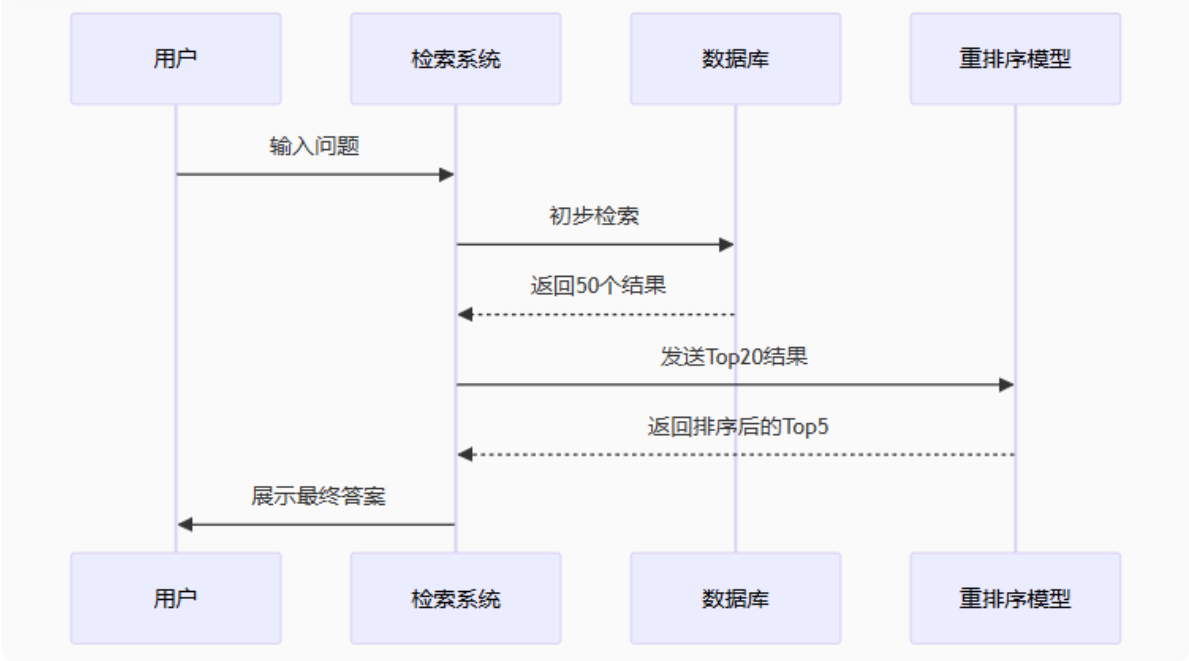
模块四：检索结果重排序

为什么要重排序？

就像面试筛选简历：

- 1. 初筛：快速浏览100份简历（初步检索）
- 2. 精筛：详细评估前20份（重排序）
- 3. 终选：确定3位候选人（最终结果）

重排序工作流程：



常见排序模型对比：

模型名称	速度	精度	硬件要求	适用场景
BM25	快	中	低	关键词匹配
Cross-Encoder	慢	高	高	小规模精准排序
ColBERT	中	高	中	平衡速度与精度

无排序 vs Cohere Reranker

```
# 初始检索结果（按相似度排序）：
results = [
    "模型正则化方法简述",      # 相关度0.7
    "硬件加速技术进展",        # 相关度0.65
    "过拟合解决方案详解",      # 相关度0.92 ← 正确答案
    "数据集清洗方法"
]

# 应用重排序
from llama_index.postprocessor.cohere_rerank import CohereRerank
reranker = CohereRerank(api_key="YOUR_KEY", top_n=2)
reranked_results = reranker.postprocess_nodes(results, query_str=query)

print("重排序后结果: ", [res.text for res in reranked_results])
```

排序变化对比:

原始排序:

- 1. 模型正则化方法简述 (相关度0.7)
- 2. 硬件加速技术进展 (相关度0.65)
- 3. 过拟合解决方案详解 (相关度0.92) ← 正确答案
- 4. 数据集清洗方法

重排序后:

- 1. 过拟合解决方案详解 (评分0.95) ← 正确答案
- 2. 模型正则化方法简述 (评分0.88)